Mini-Projekt: Fabian Oppermann, Philipp Hasel, Petruta-Denisa Biholari

Punkt 2: MLPClassifier mit GridSearchCV

Klassifikation mittels MLPClassifier und Hyperparameter-Optimierung durch GridSearchCV. Visualisierung der Lernkurven (Verlust und Genauigkeit) für Training und Validierung nach jeder Epoche.

```python
# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# Daten laden
df = pd.read_csv('Most Streamed Spotify Songs 2024.csv',
encoding='latin-1')

# Überblick über die Daten
print("Datensatz-Form:", df.shape)
print("\nErste Zeilen:")
print(df.head())
print("\nDatentypen:")
print(df.dtypes)
print("\nFehlende Werte:")
print(df.isnull().sum())
```

```
Datensatz-Form: (4600, 29)

Erste Zeilen:
                        Track                     Album Name
Artist  \
0         MILLION DOLLAR BABY  Million Dollar Baby - Single   Tommy
Richman
1                 Not Like Us                   Not Like Us  Kendrick
Lamar
2  i like the way you kiss me    I like the way you kiss me
Artemas
3                     Flowers              Flowers - Single     Miley
Cyrus
4                     Houdini                       Houdini
Eminem

  Release Date          ISRC All Time Rank  Track Score Spotify
Streams  \
0    4/26/2024  QM24S2402528             1        725.4
```

```
390,470,936
1     5/4/2024  USUG12400910             2        545.9
323,703,884
2    3/19/2024  QZJ842400387             3        538.4
601,309,283
3    1/12/2023  USSM12209777             4        444.9
2,031,280,633
4    5/31/2024  USUG12403398             5        423.3
107,034,922

  Spotify Playlist Count Spotify Playlist Reach  ...  SiriusXM
Spins  \
0                 30,716            196,631,588  ...             684

1                 28,113            174,597,137  ...               3

2                 54,331            211,607,669  ...             536

3                269,802            136,569,078  ...           2,182

4                  7,223            151,469,874  ...               1


  Deezer Playlist Count Deezer Playlist Reach Amazon Playlist Count  \
0                  62.0            17,598,718                 114.0
1                  67.0            10,422,430                 111.0
2                 136.0            36,321,847                 172.0
3                 264.0            24,684,248                 210.0
4                  82.0            17,660,624                 105.0

  Pandora Streams Pandora Track Stations Soundcloud Streams  Shazam
Counts  \
0      18,004,655                 22,931          4,818,457
2,669,262
1       7,780,028                 28,444          6,623,075
1,118,279
2       5,022,621                  5,639          7,208,651
5,285,340
3     190,260,277                203,384                NaN
11,822,942
4       4,493,884                  7,006            207,179
457,017

  TIDAL Popularity Explicit Track
0              NaN              0
1              NaN              1
2              NaN              0
3              NaN              0
4              NaN              1
```

```
[5 rows x 29 columns]

Datentypen:
Track                                 str
Album Name                            str
Artist                                str
Release Date                          str
ISRC                                  str
All Time Rank                         str
Track Score                       float64
Spotify Streams                       str
Spotify Playlist Count                str
Spotify Playlist Reach                str
Spotify Popularity                float64
YouTube Views                         str
YouTube Likes                         str
TikTok Posts                          str
TikTok Likes                          str
TikTok Views                          str
YouTube Playlist Reach                str
Apple Music Playlist Count        float64
AirPlay Spins                         str
SiriusXM Spins                        str
Deezer Playlist Count             float64
Deezer Playlist Reach                 str
Amazon Playlist Count             float64
Pandora Streams                       str
Pandora Track Stations                str
Soundcloud Streams                    str
Shazam Counts                         str
TIDAL Popularity                  float64
Explicit Track                      int64
dtype: object

Fehlende Werte:
Track                                 0
Album Name                            0
Artist                                5
Release Date                          0
ISRC                                  0
All Time Rank                         0
Track Score                           0
Spotify Streams                     113
Spotify Playlist Count               70
Spotify Playlist Reach               72
Spotify Popularity                  804
YouTube Views                       308
YouTube Likes                       315
TikTok Posts                       1173
```

```
TikTok Likes                    980
TikTok Views                    981
YouTube Playlist Reach         1009
Apple Music Playlist Count      561
AirPlay Spins                   498
SiriusXM Spins                 2123
Deezer Playlist Count           921
Deezer Playlist Reach           928
Amazon Playlist Count          1055
Pandora Streams                1106
Pandora Track Stations         1268
Soundcloud Streams             3333
Shazam Counts                   577
TIDAL Popularity               4600
Explicit Track                    0
dtype: int64
```

```python
# Datenvorverarbeitung
# Ziel: Klassifikation von 'Explicit Track' (explizit vs. nicht explizit)

# Numerische Spalten auswählen und fehlende Werte behandeln
numeric_cols = ['Spotify Streams', 'Spotify Playlist Count', 'Spotify Popularity',
                'YouTube Views', 'YouTube Likes', 'TikTok Posts', 'TikTok Likes',
                'TikTok Views', 'Shazam Counts']

# Kommas aus Zahlenwerten entfernen und in float konvertieren
for col in numeric_cols:
    if col in df.columns:
        df[col] = df[col].astype(str).str.replace(',', '').replace('', '0')
        df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)

# Feature-Matrix (X) und Zielvariable (y) erstellen
X = df[numeric_cols].copy()
y = df['Explicit Track'].copy()

# Fehlende Werte in y behandeln
y = y.fillna(0).astype(int)

print("Features (X) Form:", X.shape)
print("Ziel (y) Form:", y.shape)
print("Klassenverteilung:")
print(y.value_counts())
```

```
Features (X) Form: (4600, 9)
Ziel (y) Form: (4600,)
Klassenverteilung:
```

```
Explicit Track
0    2949
1    1651
Name: count, dtype: int64
```

```python
# Train-Test-Split: 80% Training, 20% Test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

# Feature Scaling: Standardisierung der numerischen Features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training Set Größe:", X_train_scaled.shape)
print("Test Set Größe:", X_test_scaled.shape)
```

```
Training Set Größe: (3680, 9)
Test Set Größe: (920, 9)
```

```python
# GridSearchCV für Hyperparameter-Optimierung
# Parameter-Grid: verschiedene Hidden Layer Strukturen,
Aktivierungsfunktionen, Solver

param_grid = {
    'hidden_layer_sizes': [(50,), (100,), (50, 50), (100, 50)],
    'activation': ['relu', 'tanh'],
    'solver': ['adam', 'sgd'],
    'alpha': [0.0001, 0.001],  # L2-Regularisierung
    'learning_rate': ['constant', 'adaptive']
}

# MLPClassifier mit frühem Stoppen (early stopping) für Validierung
mlp = MLPClassifier(max_iter=100, random_state=42,
early_stopping=True,
                    validation_fraction=0.2, n_iter_no_change=10,
verbose=False)

# GridSearchCV: 5-fache Kreuzvalidierung
grid_search = GridSearchCV(estimator=mlp, param_grid=param_grid,
                           cv=5, scoring='accuracy', n_jobs=-1,
verbose=2)

print("GridSearchCV gestartet...")
grid_search.fit(X_train_scaled, y_train)

print("\nBeste Parameter:", grid_search.best_params_)
print("Beste Kreuzvalidierungs-Genauigkeit:",
f"{grid_search.best_score_:.4f}")
```

```
GridSearchCV gestartet...
Fitting 5 folds for each of 64 candidates, totalling 320 fits

Beste Parameter: {'activation': 'relu', 'alpha': 0.0001,
'hidden_layer_sizes': (100, 50), 'learning_rate': 'constant',
'solver': 'adam'}
Beste Kreuzvalidierungs-Genauigkeit: 0.7079
```

```python
# Bestes Modell evaluieren
best_mlp = grid_search.best_estimator_

# Vorhersage auf Test-Set
y_pred = best_mlp.predict(X_test_scaled)

# Genauigkeit berechnen
test_accuracy = accuracy_score(y_test, y_pred)
print(f"Test-Genauigkeit: {test_accuracy:.4f}")

# Detaillierter Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Test-Genauigkeit: 0.7272

Classification Report:
              precision    recall  f1-score   support

           0       0.74      0.88      0.81       590
           1       0.68      0.45      0.54       330

    accuracy                           0.73       920
   macro avg       0.71      0.67      0.67       920
weighted avg       0.72      0.73      0.71       920
```

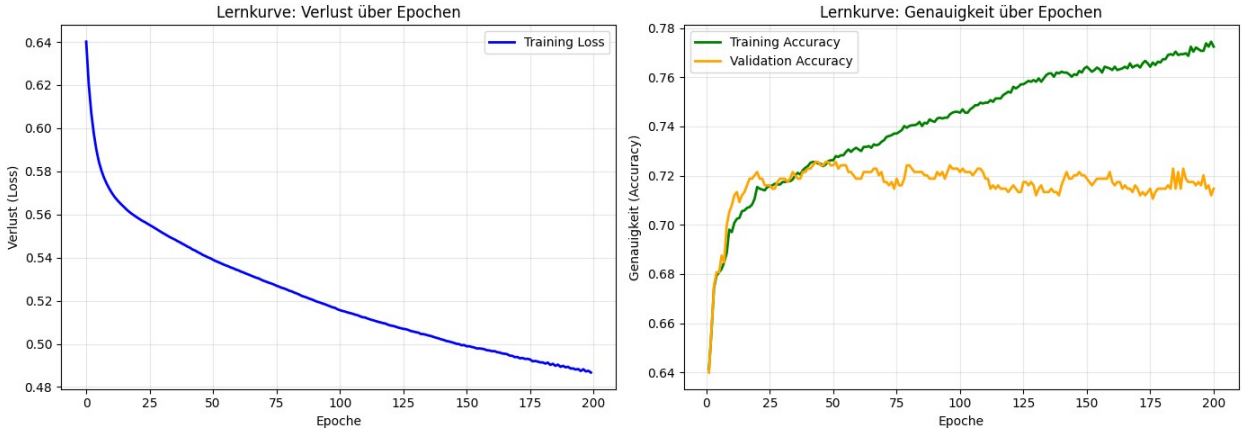
```python
import warnings
from sklearn.exceptions import ConvergenceWarning

best_params = grid_search.best_params_

# Validation Set aus Training erstellen
X_train_part, X_val, y_train_part, y_val = train_test_split(
    X_train_scaled, y_train, test_size=0.2, random_state=42
)

# Modell mit warm_start=True: dasselbe Modell wird Epoche für Epoche
weitertrainiert
mlp_for_curves = MLPClassifier(
    hidden_layer_sizes=best_params['hidden_layer_sizes'],
    activation=best_params['activation'],
    solver=best_params['solver'],
```

```
    alpha=best_params['alpha'],
    learning_rate=best_params['learning_rate'],
    max_iter=1,       # Eine Epoche pro fit()-Aufruf
    random_state=42,
    warm_start=True,  # Gewichte bleiben zwischen fit()-Aufrufen
erhalten
    tol=0.0           # Kein vorzeitiges Stoppen durch
Konvergenzkriterium
)

train_accuracies = []
val_accuracies = []

with warnings.catch_warnings():
    warnings.simplefilter("ignore", ConvergenceWarning)
    for epoch in range(200):
        mlp_for_curves.fit(X_train_part, y_train_part)
        train_accuracies.append(mlp_for_curves.score(X_train_part,
y_train_part))
        val_accuracies.append(mlp_for_curves.score(X_val, y_val))

epochs = list(range(1, 201))

print(f"Training abgeschlossen nach {len(mlp_for_curves.loss_curve_)}
Iterationen")
print(f"Finale Verlust-Funktion: {mlp_for_curves.loss_:.4f}")
print(f"Finale Training Accuracy: {train_accuracies[-1]:.4f}")
print(f"Finale Validation Accuracy: {val_accuracies[-1]:.4f}")
```

```
Training abgeschlossen nach 200 Iterationen
Finale Verlust-Funktion: 0.4867
Finale Training Accuracy: 0.7724
Finale Validation Accuracy: 0.7147
```

```
# Visualisierung der Lernkurven
plt.figure(figsize=(14, 5))

# Subplot 1: Verlust über Epochen
plt.subplot(1, 2, 1)
plt.plot(mlp_for_curves.loss_curve_, label='Training Loss',
color='blue', linewidth=2)
plt.xlabel('Epoche')
plt.ylabel('Verlust (Loss)')
plt.title('Lernkurve: Verlust über Epochen')
plt.legend()
plt.grid(True, alpha=0.3)

# Subplot 2: Genauigkeit über Epochen (pro Epoche, nicht alle 10)
plt.subplot(1, 2, 2)
plt.plot(epochs, train_accuracies, label='Training Accuracy',
```

```
color='green', linewidth=2)
plt.plot(epochs, val_accuracies, label='Validation Accuracy',
color='orange', linewidth=2)
plt.xlabel('Epoche')
plt.ylabel('Genauigkeit (Accuracy)')
plt.title('Lernkurve: Genauigkeit über Epochen')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\nFinale Training Accuracy: {train_accuracies[-1]:.4f}")
print(f"Finale Validation Accuracy: {val_accuracies[-1]:.4f}")
```




```
Finale Training Accuracy: 0.7724
Finale Validation Accuracy: 0.7147
```

Punkt 3: Klassifikation mittels Keras und KerasTuner (Hyperband)

Klassifikation von 'Explicit Track' mittels Keras Neural Network. Hyperparameter-Optimierung durch KerasTuner mit dem Hyperband-Algorithmus. Visualisierung der Lernkurven (Verlust und Genauigkeit) für Training und Validierung nach jeder Epoche.

```
# Imports für Keras und KerasTuner
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks
import keras
import keras_tuner as kt
```

```python
print("TensorFlow Version:", tf.__version__)
print("KerasTuner Version:", kt.__version__)
```

```
TensorFlow Version: 2.20.0
KerasTuner Version: 1.4.8
```

```python
# Modell-Builder-Funktion für KerasTuner
# Hyperparameter werden vom Tuner automatisch variiert

def build_model(hp):
    model = keras.Sequential()

    # Eingabeschicht
    model.add(layers.Input(shape=(X_train_scaled.shape[1],)))

    # Anzahl der Hidden Layers (1-3)
    for i in range(hp.Int('num_layers', min_value=1, max_value=3,
default=2)):
        units = hp.Int(f'units_{i}', min_value=32, max_value=256,
step=32, default=128)
        model.add(layers.Dense(units=units))

        # Aktivierungsfunktion
        activation = hp.Choice('activation', values=['relu', 'tanh',
'selu'], default='relu')
        model.add(layers.Activation(activation))

        # Dropout-Rate
        dropout_rate = hp.Float(f'dropout_{i}', min_value=0.0,
max_value=0.5, step=0.1, default=0.2)
        model.add(layers.Dropout(rate=dropout_rate))

    # Ausgabeschicht (binäre Klassifikation)
    model.add(layers.Dense(1, activation='sigmoid'))

    # Lernrate
    learning_rate = hp.Float('learning_rate', min_value=1e-4,
max_value=1e-2, sampling='log', default=1e-3)

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

# Test: Modell mit Standardparametern erstellen
test_model = build_model(kt.HyperParameters())
test_model.summary()
```

```
Model: "sequential"

┏━━━
```

```
    directory='keras_tuner_dir',
    project_name='spotify_classification',
    overwrite=False
)

# Zusammenfassung des Suchraums
tuner.search_space_summary()

# Early Stopping Callback für die Suche
early_stop = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

# Hyperparameter-Suche starten
print("\nHyperband-Suche gestartet...")
tuner.search(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

# Beste Hyperparameter anzeigen
print("\n--- Beste Hyperparameter ---")
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
print(f"Anzahl Layers: {best_hps.get('num_layers')}")
for i in range(best_hps.get('num_layers')):
    print(f"  Layer {i+1} Units: {best_hps.get(f'units_{i}')}")
    print(f"  Layer {i+1} Dropout: {best_hps.get(f'dropout_{i}')}")
print(f"Aktivierung: {best_hps.get('activation')}")
print(f"Learning Rate: {best_hps.get('learning_rate'):.6f}")
```

```
Reloading Tuner from keras_tuner_dir\spotify_classification\
tuner0.json
Search space summary
Default search space size: 9
num_layers (Int)
{'default': 2, 'conditions': [], 'min_value': 1, 'max_value': 3,
'step': 1, 'sampling': 'linear'}
units_0 (Int)
{'default': 128, 'conditions': [], 'min_value': 32, 'max_value': 256,
'step': 32, 'sampling': 'linear'}
activation (Choice)
{'default': 'relu', 'conditions': [], 'values': ['relu', 'tanh',
'selu'], 'ordered': False}
dropout_0 (Float)
```

```
{'default': 0.2, 'conditions': [], 'min_value': 0.0, 'max_value': 0.5,
'step': 0.1, 'sampling': 'linear'}
units_1 (Int)
{'default': 128, 'conditions': [], 'min_value': 32, 'max_value': 256,
'step': 32, 'sampling': 'linear'}
dropout_1 (Float)
{'default': 0.2, 'conditions': [], 'min_value': 0.0, 'max_value': 0.5,
'step': 0.1, 'sampling': 'linear'}
learning_rate (Float)
{'default': 0.001, 'conditions': [], 'min_value': 0.0001, 'max_value':
0.01, 'step': None, 'sampling': 'log'}
units_2 (Int)
{'default': 128, 'conditions': [], 'min_value': 32, 'max_value': 256,
'step': 32, 'sampling': 'linear'}
dropout_2 (Float)
{'default': 0.2, 'conditions': [], 'min_value': 0.0, 'max_value': 0.5,
'step': 0.1, 'sampling': 'linear'}

Hyperband-Suche gestartet...

--- Beste Hyperparameter ---
Anzahl Layers: 3
  Layer 1 Units: 224
  Layer 1 Dropout: 0.2
  Layer 2 Units: 64
  Layer 2 Dropout: 0.0
  Layer 3 Units: 64
  Layer 3 Dropout: 0.2
Aktivierung: relu
Learning Rate: 0.000901
```

```
# Bestes Modell mit den optimalen Hyperparametern neu trainieren

best_model = tuner.hypermodel.build(best_hps)

# Callbacks: EarlyStopping + TensorBoard
tensorboard_cb = callbacks.TensorBoard(
    log_dir='./logs/keras_tuner_best',
    histogram_freq=1
)

early_stop_final = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

# Training des besten Modells mit Validierungssplit
import time as _time_module
print("Training des besten Modells...")
```

```
_keras_train_start = _time_module.time()
history = best_model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=[early_stop_final, tensorboard_cb],
    verbose=1
)
keras_train_duration = _time_module.time() - _keras_train_start

print(f"\nTraining beendet nach {len(history.history['loss'])}
Epochen")
print(f"Beste Validierungs-Genauigkeit:
{max(history.history['val_accuracy']):.4f}")
print(f"Bester Validierungs-Verlust:
{min(history.history['val_loss']):.4f}")
```

```
Training des besten Modells...
Epoch 1/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.6413 - loss:
0.6331 - val_accuracy: 0.6821 - val_loss: 0.5977
Epoch 2/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.6831 - loss:
0.5985 - val_accuracy: 0.7011 - val_loss: 0.5718
Epoch 3/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.6936 - loss:
0.5850 - val_accuracy: 0.6902 - val_loss: 0.5660
Epoch 4/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.6974 - loss:
0.5770 - val_accuracy: 0.6997 - val_loss: 0.5786
Epoch 5/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.6957 - loss:
0.5763 - val_accuracy: 0.7120 - val_loss: 0.5515
Epoch 6/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7014 - loss:
0.5716 - val_accuracy: 0.7079 - val_loss: 0.5558
Epoch 7/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7123 - loss:
0.5658 - val_accuracy: 0.7079 - val_loss: 0.5498
Epoch 8/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7106 - loss:
0.5614 - val_accuracy: 0.7079 - val_loss: 0.5474
Epoch 9/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7082 - loss:
0.5610 - val_accuracy: 0.7120 - val_loss: 0.5765
Epoch 10/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7116 - loss:
0.5581 - val_accuracy: 0.7174 - val_loss: 0.5502
Epoch 11/100
```

```
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7092 - loss:
0.5600 - val_accuracy: 0.7201 - val_loss: 0.5421
Epoch 12/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7171 - loss:
0.5567 - val_accuracy: 0.7092 - val_loss: 0.5488
Epoch 13/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7154 - loss:
0.5532 - val_accuracy: 0.7147 - val_loss: 0.5395
Epoch 14/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7160 - loss:
0.5493 - val_accuracy: 0.7120 - val_loss: 0.5510
Epoch 15/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7289 - loss:
0.5447 - val_accuracy: 0.7255 - val_loss: 0.5356
Epoch 16/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7191 - loss:
0.5431 - val_accuracy: 0.7147 - val_loss: 0.5546
Epoch 17/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7204 - loss:
0.5439 - val_accuracy: 0.7242 - val_loss: 0.5363
Epoch 18/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7303 - loss:
0.5401 - val_accuracy: 0.7106 - val_loss: 0.5460
Epoch 19/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7188 - loss:
0.5397 - val_accuracy: 0.7323 - val_loss: 0.5366
Epoch 20/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7255 - loss:
0.5361 - val_accuracy: 0.7147 - val_loss: 0.5454
Epoch 21/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7347 - loss:
0.5313 - val_accuracy: 0.7283 - val_loss: 0.5335
Epoch 22/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7276 - loss:
0.5351 - val_accuracy: 0.7215 - val_loss: 0.5399
Epoch 23/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7296 - loss:
0.5297 - val_accuracy: 0.7215 - val_loss: 0.5338
Epoch 24/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7286 - loss:
0.5289 - val_accuracy: 0.7242 - val_loss: 0.5308
Epoch 25/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7344 - loss:
0.5239 - val_accuracy: 0.7228 - val_loss: 0.5315
Epoch 26/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7337 - loss:
0.5259 - val_accuracy: 0.7092 - val_loss: 0.5350
Epoch 27/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7323 - loss:
```

```
0.5260 - val_accuracy: 0.7269 - val_loss: 0.5260
Epoch 28/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7432 - loss:
0.5216 - val_accuracy: 0.7201 - val_loss: 0.5479
Epoch 29/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7371 - loss:
0.5222 - val_accuracy: 0.7228 - val_loss: 0.5315
Epoch 30/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7385 - loss:
0.5148 - val_accuracy: 0.7296 - val_loss: 0.5248
Epoch 31/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7381 - loss:
0.5203 - val_accuracy: 0.7065 - val_loss: 0.5473
Epoch 32/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7429 - loss:
0.5126 - val_accuracy: 0.7228 - val_loss: 0.5290
Epoch 33/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7398 - loss:
0.5184 - val_accuracy: 0.7323 - val_loss: 0.5247
Epoch 34/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7415 - loss:
0.5151 - val_accuracy: 0.7133 - val_loss: 0.5339
Epoch 35/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7354 - loss:
0.5189 - val_accuracy: 0.7120 - val_loss: 0.5311
Epoch 36/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7398 - loss:
0.5101 - val_accuracy: 0.7242 - val_loss: 0.5279
Epoch 37/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7418 - loss:
0.5088 - val_accuracy: 0.7160 - val_loss: 0.5382
Epoch 38/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7432 - loss:
0.5064 - val_accuracy: 0.7201 - val_loss: 0.5338
Epoch 39/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7493 - loss:
0.5070 - val_accuracy: 0.7269 - val_loss: 0.5231
Epoch 40/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7446 - loss:
0.5047 - val_accuracy: 0.7269 - val_loss: 0.5207
Epoch 41/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7473 - loss:
0.5017 - val_accuracy: 0.7106 - val_loss: 0.5467
Epoch 42/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7527 - loss:
0.5050 - val_accuracy: 0.7174 - val_loss: 0.5306
Epoch 43/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7507 - loss:
0.5008 - val_accuracy: 0.7160 - val_loss: 0.5277
```

```
Epoch 44/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7490 - loss:
0.5035 - val_accuracy: 0.7188 - val_loss: 0.5355
Epoch 45/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7561 - loss:
0.5021 - val_accuracy: 0.7052 - val_loss: 0.5488
Epoch 46/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7588 - loss:
0.4992 - val_accuracy: 0.7092 - val_loss: 0.5455
Epoch 47/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7531 - loss:
0.4962 - val_accuracy: 0.7201 - val_loss: 0.5316
Epoch 48/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7605 - loss:
0.4925 - val_accuracy: 0.7079 - val_loss: 0.5425
Epoch 49/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7612 - loss:
0.4957 - val_accuracy: 0.7283 - val_loss: 0.5253
Epoch 50/100
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7582 - loss:
0.4925 - val_accuracy: 0.7133 - val_loss: 0.5346

Training beendet nach 50 Epochen
Beste Validierungs-Genauigkeit: 0.7323
Bester Validierungs-Verlust: 0.5207
```

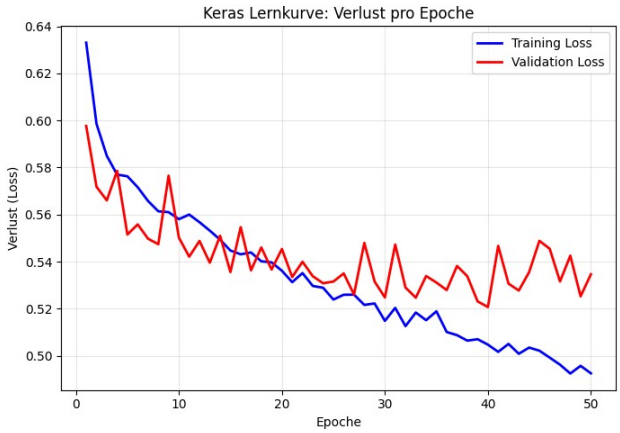
```python
# Evaluation auf dem Test-Set
from sklearn.metrics import classification_report, confusion_matrix

# Vorhersage
y_pred_keras_prob = best_model.predict(X_test_scaled)
y_pred_keras = (y_pred_keras_prob > 0.5).astype(int).flatten()

# Genauigkeit
test_loss, test_accuracy = best_model.evaluate(X_test_scaled, y_test,
verbose=0)
print(f"Keras Test-Verlust: {test_loss:.4f}")
print(f"Keras Test-Genauigkeit: {test_accuracy:.4f}")

# Classification Report
print("\nClassification Report (Keras):")
print(classification_report(y_test, y_pred_keras))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_keras)
print("Confusion Matrix:")
print(cm)
```

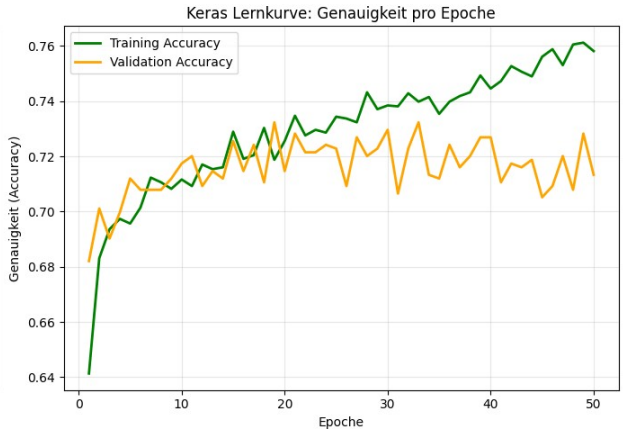
```
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
Keras Test-Verlust: 0.5363
```

```
Keras Test-Genauigkeit: 0.7272

Classification Report (Keras):
              precision    recall  f1-score   support

           0       0.75      0.85      0.80       590
           1       0.66      0.50      0.57       330

    accuracy                           0.73       920
   macro avg       0.71      0.68      0.68       920
weighted avg       0.72      0.73      0.72       920

Confusion Matrix:
[[504  86]
 [165 165]]
```

```python
# Visualisierung der Lernkurven: Verlust und Genauigkeit pro Epoche
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

epochs_range = range(1, len(history.history['loss']) + 1)

# --- Subplot 1: Durchschnittlicher Verlust ---
axes[0].plot(epochs_range, history.history['loss'], label='Training
Loss', color='blue', linewidth=2)
axes[0].plot(epochs_range, history.history['val_loss'],
label='Validation Loss', color='red', linewidth=2)
axes[0].set_xlabel('Epoche')
axes[0].set_ylabel('Verlust (Loss)')
axes[0].set_title('Keras Lernkurve: Verlust pro Epoche')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# --- Subplot 2: Durchschnittliche Genauigkeit ---
axes[1].plot(epochs_range, history.history['accuracy'],
label='Training Accuracy', color='green', linewidth=2)
axes[1].plot(epochs_range, history.history['val_accuracy'],
label='Validation Accuracy', color='orange', linewidth=2)
axes[1].set_xlabel('Epoche')
axes[1].set_ylabel('Genauigkeit (Accuracy)')
axes[1].set_title('Keras Lernkurve: Genauigkeit pro Epoche')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Zusammenfassung
print(f"Finale Training Accuracy: {history.history['accuracy'][-
1]:.4f}")
print(f"Finale Validation Accuracy: {history.history['val_accuracy'][-
```

```
1]:.4f}")
print(f"Finale Training Loss: {history.history['loss'][-1]:.4f}")
print(f"Finale Validation Loss: {history.history['val_loss'][-
1]:.4f}")
```






```
Finale Training Accuracy: 0.7582
Finale Validation Accuracy: 0.7133
Finale Training Loss: 0.4925
Finale Validation Loss: 0.5346
```

Punkt 4: Vergleichstabelle & Diskussion

Hier vergleichen wir die neuronalen Netze aus Punkt 2 und 3 mit den klassischen ML-Modellen aus der 4. Klasse. Damit es fair bleibt, laufen alle auf den gleichen Daten und dem gleichen Split. Wir schauen uns Accuracy, Trainingszeit und Inferenzzeit an.

4.1 Klassische ML-Modelle aus der 4. Klasse

In der 4. Klasse hatten wir eine andere Zielvariable (`PopularityClass`), deshalb können wir die alten Ergebnisse nicht direkt übernehmen. Wir trainieren die 5 Algorithmen hier nochmal auf denselben Daten wie in Punkt 2 und 3 (`Explicit Track`).

```
import time
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

classifiers_4th = {
    'Logistic Regression': LogisticRegression(max_iter=1000, C=1.0,
solver='liblinear', random_state=42),
    'SVM (RBF)': SVC(kernel='rbf', C=1.0, random_state=42),
```

```
    'Decision Tree': DecisionTreeClassifier(max_depth=10,
min_samples_split=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100,
max_depth=10, random_state=42),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5,
weights='distance')
}

results_4th = {}

print("Training der klassischen ML-Modelle aus der 4. Klasse:\n")
for name, clf in classifiers_4th.items():
    start_train = time.time()
    clf.fit(X_train_scaled, y_train)
    train_time = time.time() - start_train

    start_pred = time.time()
    y_pred = clf.predict(X_test_scaled)
    pred_time = time.time() - start_pred

    train_acc = clf.score(X_train_scaled, y_train)
    test_acc = accuracy_score(y_test, y_pred)

    results_4th[name] = {
        'Train Accuracy': round(train_acc, 4),
        'Test Accuracy': round(test_acc, 4),
        'Trainingszeit (s)': round(train_time, 4),
        'Inferenzzeit (s)': round(pred_time, 4)
    }

    print(f"--- {name} ---")
    print(f"  Train Accuracy:   {train_acc:.4f}")
    print(f"  Test Accuracy:    {test_acc:.4f}")
    print(f"  Trainingszeit:    {train_time:.4f} s")
    print(f"  Inferenzzeit:     {pred_time:.4f} s\n")
```

```
Training der klassischen ML-Modelle aus der 4. Klasse:

--- Logistic Regression ---
  Train Accuracy:   0.6826
  Test Accuracy:    0.6935
  Trainingszeit:    0.0153 s
  Inferenzzeit:     0.0010 s

--- SVM (RBF) ---
  Train Accuracy:   0.7204
  Test Accuracy:    0.7207
  Trainingszeit:    0.5758 s
  Inferenzzeit:     0.3266 s
```

```
--- Decision Tree ---
  Train Accuracy:   0.8476
  Test Accuracy:    0.6826
  Trainingszeit:    0.0452 s
  Inferenzzeit:     0.0010 s

--- Random Forest ---
  Train Accuracy:   0.9234
  Test Accuracy:    0.7261
  Trainingszeit:    1.2697 s
  Inferenzzeit:     0.0347 s

--- KNN (k=5) ---
  Train Accuracy:   0.9997
  Test Accuracy:    0.6913
  Trainingszeit:    0.0084 s
  Inferenzzeit:     0.0334 s
```

4.2 Ergebnisse aus Punkt 2 und 3

Die Accuracy und Vorhersagen kommen direkt von den schon trainierten Modellen `best_mlp` und `best_model`. Trainingszeit für den MLP kommt aus `grid_search.refit_time_`, für Keras aus der gemessenen Trainingszeit.

```
print("Beste MLP-Parameter aus GridSearchCV:",
grid_search.best_params_)

train_acc_mlp = best_mlp.score(X_train_scaled, y_train)
y_pred_mlp = best_mlp.predict(X_test_scaled)
test_acc_mlp = accuracy_score(y_test, y_pred_mlp)

train_time_mlp = grid_search.refit_time_

start_pred_mlp = time.time()
_ = best_mlp.predict(X_test_scaled)
pred_time_mlp = time.time() - start_pred_mlp

print(f"\n--- MLPClassifier (Punkt 2) ---")
print(f"  Train Accuracy:   {train_acc_mlp:.4f}")
print(f"  Test Accuracy:    {test_acc_mlp:.4f}")
print(f"  Trainingszeit:    {train_time_mlp:.4f} s")
print(f"  Inferenzzeit:     {pred_time_mlp:.4f} s")

print("\n\nBeste Keras-Hyperparameter aus KerasTuner:")
print(f"  Layers: {best_hps.get('num_layers')}, Activation:
{best_hps.get('activation')}, LR:
{best_hps.get('learning_rate'):.6f}")

y_pred_keras = (best_model.predict(X_test_scaled, verbose=0) >
```

```
0.5).astype(int).flatten()
train_acc_keras = accuracy_score(y_train,
(best_model.predict(X_train_scaled, verbose=0) >
0.5).astype(int).flatten())
test_acc_keras = accuracy_score(y_test, y_pred_keras)

train_time_keras = keras_train_duration

start_pred_keras = time.time()
_ = best_model.predict(X_test_scaled, verbose=0)
pred_time_keras = time.time() - start_pred_keras

print(f"\n--- Keras + KerasTuner (Punkt 3) ---")
print(f"  Train Accuracy:   {train_acc_keras:.4f}")
print(f"  Test Accuracy:    {test_acc_keras:.4f}")
print(f"  Trainingszeit:    {train_time_keras:.4f} s (geschätzt aus
Epochenanzahl)")
print(f"  Trainingszeit:    {train_time_keras:.4f} s")
```

```
Beste MLP-Parameter aus GridSearchCV: {'activation': 'relu', 'alpha':
0.0001, 'hidden_layer_sizes': (100, 50), 'learning_rate': 'constant',
'solver': 'adam'}

--- MLPClassifier (Punkt 2) ---
  Train Accuracy:   0.7372
  Test Accuracy:    0.7272
  Trainingszeit:    1.4569 s
  Inferenzzeit:     0.0030 s


Beste Keras-Hyperparameter aus KerasTuner:
  Layers: 3, Activation: relu, LR: 0.000901

--- Keras + KerasTuner (Punkt 3) ---
  Train Accuracy:   0.7579
  Test Accuracy:    0.7272
  Trainingszeit:    39.8246 s (geschätzt aus Epochenanzahl)
  Trainingszeit:    39.8246 s
```

4.3 Vergleichstabelle

Alle Ergebnisse in einer Tabelle zusammengefasst, sortiert nach Test-Accuracy.

```
all_results = {}

for name, res in results_4th.items():
    all_results[name + ' (4. Klasse)'] = res

all_results['MLPClassifier + GridSearchCV (Punkt 2)'] = {
    'Train Accuracy': round(train_acc_mlp, 4),
```

```
    'Test Accuracy': round(test_acc_mlp, 4),
    'Trainingszeit (s)': round(train_time_mlp, 4),
    'Inferenzzeit (s)': round(pred_time_mlp, 4)
}

all_results['Keras + KerasTuner (Punkt 3)'] = {
    'Train Accuracy': round(train_acc_keras, 4),
    'Test Accuracy': round(test_acc_keras, 4),
    'Trainingszeit (s)': round(train_time_keras, 4),
    'Inferenzzeit (s)': round(pred_time_keras, 4)
}

comparison_df = pd.DataFrame(all_results).T
comparison_df = comparison_df.sort_values(by='Test Accuracy',
ascending=False)
comparison_df.index.name = 'Modell'

print("=" * 90)
print("VERGLEICHSTABELLE: Alle Modelle im Überblick")
print("=" * 90)
print(comparison_df.to_string())
print("=" * 90)

comparison_df.style.highlight_max(subset=['Test Accuracy'],
color='lightgreen') \
                    .highlight_min(subset=['Trainingszeit (s)',
'Inferenzzeit (s)'], color='lightblue') \
                    .format(precision=4)
```

```
==========================================================================
====================
VERGLEICHSTABELLE: Alle Modelle im Überblick
==========================================================================
====================
                                          Train Accuracy  Test Accuracy
Trainingszeit (s)  Inferenzzeit (s)
Modell

MLPClassifier + GridSearchCV (Punkt 2)          0.7372         0.7272
1.4569            0.0030
Keras + KerasTuner (Punkt 3)                    0.7579         0.7272
39.8246            0.2050
Random Forest (4. Klasse)                       0.9234         0.7261
1.2697            0.0347
SVM (RBF) (4. Klasse)                           0.7204         0.7207
0.5758            0.3266
Logistic Regression (4. Klasse)                 0.6826         0.6935
0.0153            0.0010
KNN (k=5) (4. Klasse)                           0.9997         0.6913
0.0084            0.0334
```

```
Decision Tree (4. Klasse)                        0.8476         0.6826
0.0452            0.0010
=========================================================================
====================

<pandas.io.formats.style.Styler at 0x22d97a9bc20>
```

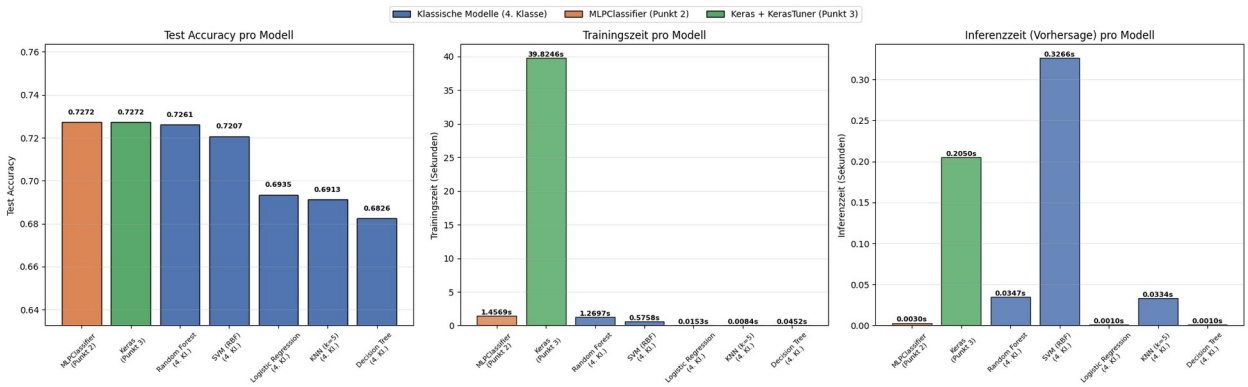
4.4 Visualisierung

Die Ergebnisse als Balkendiagramme: Blau = Modelle aus der 4. Klasse, Orange = MLP (Punkt 2), Grün = Keras.

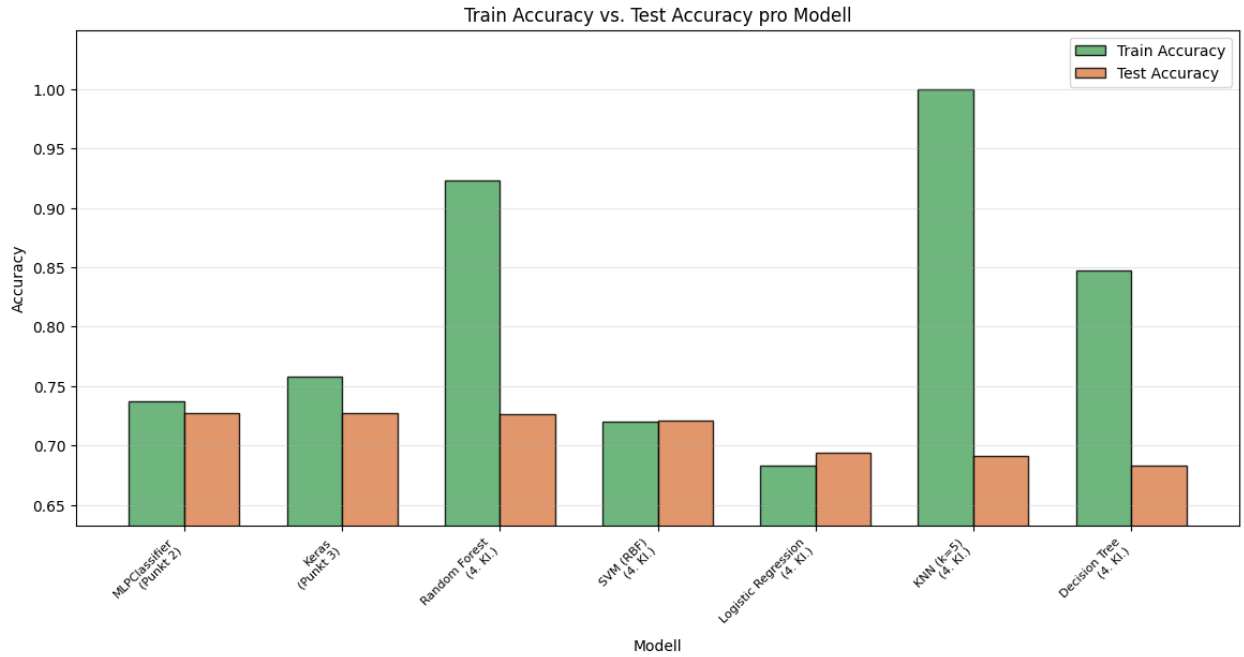
```
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

short_names = [name.replace(' (4. Klasse)', '\n(4. Kl.)').replace(' +
GridSearchCV (Punkt 2)', '\n(Punkt 2)').replace(' + KerasTuner (Punkt
3)', '\n(Punkt 3)') for name in comparison_df.index]

colors = []
for name in comparison_df.index:
    if '4. Klasse' in name:
        colors.append('#4C72B0')
    elif 'Punkt 2' in name:
        colors.append('#DD8452')
    else:
        colors.append('#55A868')

# --- Plot 1: Test Accuracy ---
bars1 = axes[0].bar(short_names, comparison_df['Test Accuracy'],
color=colors, edgecolor='black', alpha=0.85)
axes[0].set_ylabel('Test Accuracy')
bars1 = axes[0].bar(short_names, comparison_df['Test Accuracy'],
color=colors, edgecolor='black', alpha=0.85)
axes[0].set_ylabel('Test Accuracy')
axes[0].set_title('Test Accuracy pro Modell')
axes[0].set_ylim(bottom=max(0, comparison_df['Test Accuracy'].min() -
0.05))
axes[0].grid(axis='y', alpha=0.3)
for bar, val in zip(bars1, comparison_df['Test Accuracy']):
    axes[0].text(bar.get_x() + bar.get_width()/2., bar.get_height() +
0.003,
                 f'{val:.4f}', ha='center', va='bottom', fontsize=8,
fontweight='bold')
axes[0].tick_params(axis='x', rotation=45, labelsize=8)

# --- Plot 2: Trainingszeit ---
bars2 = axes[1].bar(short_names, comparison_df['Trainingszeit (s)'],
color=colors, edgecolor='black', alpha=0.85)
axes[1].set_ylabel('Trainingszeit (Sekunden)')
```

```
axes[1].set_title('Trainingszeit pro Modell')
axes[1].grid(axis='y', alpha=0.3)
for bar, val in zip(bars2, comparison_df['Trainingszeit (s)']):
    axes[1].text(bar.get_x() + bar.get_width()/2., bar.get_height() +
0.001,
                 f'{val:.4f}s', ha='center', va='bottom', fontsize=8,
fontweight='bold')
axes[1].tick_params(axis='x', rotation=45, labelsize=8)

# --- Plot 3: Inferenzzeit ---
bars3 = axes[2].bar(short_names, comparison_df['Inferenzzeit (s)'],
color=colors, edgecolor='black', alpha=0.85)
axes[2].set_ylabel('Inferenzzeit (Sekunden)')
axes[2].set_title('Inferenzzeit (Vorhersage) pro Modell')
axes[2].grid(axis='y', alpha=0.3)
for bar, val in zip(bars3, comparison_df['Inferenzzeit (s)']):
    axes[2].text(bar.get_x() + bar.get_width()/2., bar.get_height() +
0.0001,
                 f'{val:.4f}s', ha='center', va='bottom', fontsize=8,
fontweight='bold')
axes[2].tick_params(axis='x', rotation=45, labelsize=8)

# Legende für Farbkategorien
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor='#4C72B0', edgecolor='black',
label='Klassische Modelle (4. Klasse)'),
                   Patch(facecolor='#DD8452', edgecolor='black',
label='MLPClassifier (Punkt 2)'),
                   Patch(facecolor='#55A868', edgecolor='black',
label='Keras + KerasTuner (Punkt 3)')]
fig.legend(handles=legend_elements, loc='upper center', ncol=3,
fontsize=10, bbox_to_anchor=(0.5, 1.05))

plt.tight_layout()
plt.show()

# Zusätzlich: Train vs. Test Accuracy als Grouped-Bar-Chart
fig2, ax2 = plt.subplots(figsize=(14, 6))

x = np.arange(len(comparison_df))
width = 0.35

bars_train = ax2.bar(x - width/2, comparison_df['Train Accuracy'],
width, label='Train Accuracy',
                     color='#55A868', edgecolor='black', alpha=0.85)
bars_test = ax2.bar(x + width/2, comparison_df['Test Accuracy'],
width, label='Test Accuracy',
                    color='#DD8452', edgecolor='black', alpha=0.85)

ax2.set_xlabel('Modell')
```

```
ax2.set_ylabel('Accuracy')
ax2.set_title('Train Accuracy vs. Test Accuracy pro Modell')
ax2.set_xticks(x)
ax2.set_xticklabels(short_names, rotation=45, ha='right', fontsize=8)
ax2.legend()
ax2.grid(axis='y', alpha=0.3)
ax2.set_ylim(bottom=max(0, comparison_df[['Train Accuracy', 'Test
Accuracy']].min().min() - 0.05))
```




```
(0.6325999999999999, 1.049685)
```



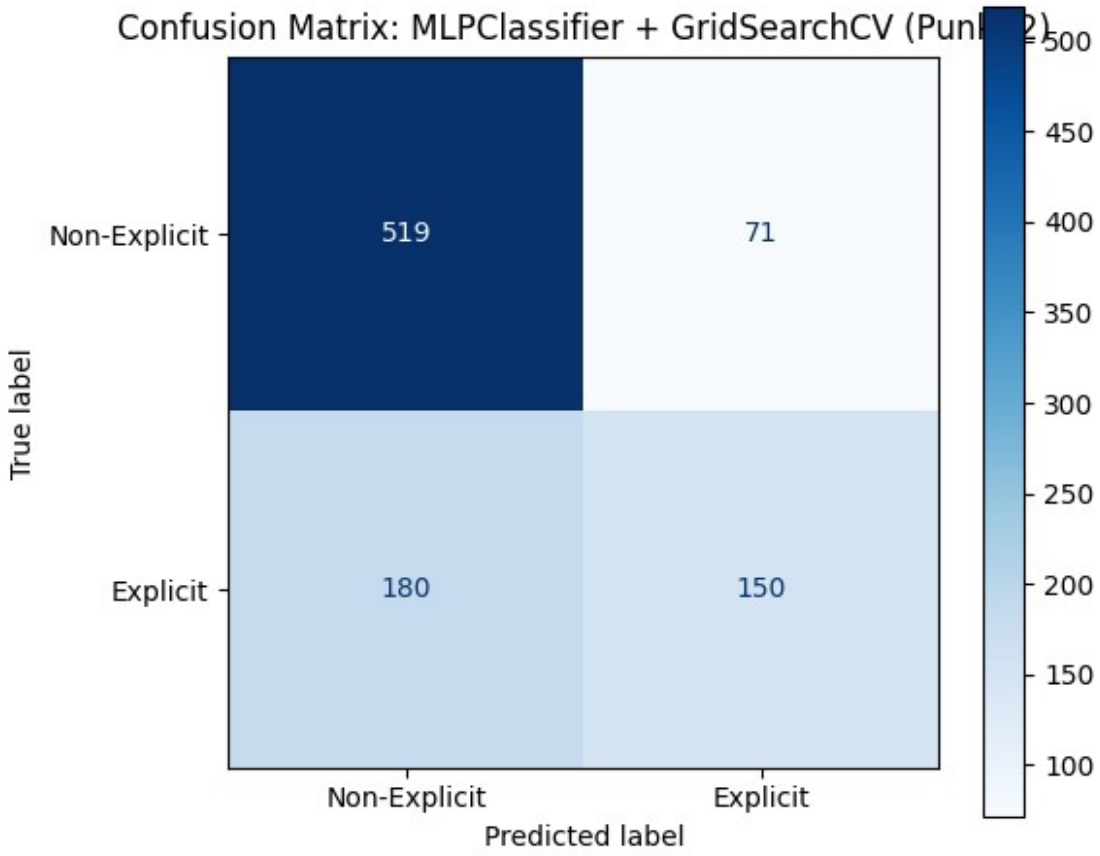
4.5 Bestes Modell genauer anschauen

Hier schauen wir uns das beste Modell nochmal mit Classification Report und Confusion Matrix an.

```python
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay

best_model_name = comparison_df['Test Accuracy'].idxmax()
print(f"Bestes Modell: {best_model_name}")
print(f"Test Accuracy: {comparison_df.loc[best_model_name, 'Test
Accuracy']:.4f}")
print(f"Trainingszeit: {comparison_df.loc[best_model_name,
'Trainingszeit (s)']:.4f} s")
print(f"Inferenzzeit:  {comparison_df.loc[best_model_name,
'Inferenzzeit (s)']:.4f} s")

# Vorhersage des besten Modells nochmal bekommen und detailliert
auswerten
all_predictions = {}

for name, clf in classifiers_4th.items():
    full_name = name + ' (4. Klasse)'
    all_predictions[full_name] = clf.predict(X_test_scaled)

# MLP Predictions (vom Original-Modell aus Punkt 2)
all_predictions['MLPClassifier + GridSearchCV (Punkt 2)'] =
best_mlp.predict(X_test_scaled)

all_predictions['Keras + KerasTuner (Punkt 3)'] =
(best_model.predict(X_test_scaled, verbose=0) >
0.5).astype(int).flatten()

y_pred_best = all_predictions[best_model_name]
# Classification Report für das beste Modell
print(f"\n{'=' * 60}")
print(f"Classification Report für {best_model_name}:")
print(f"{'=' * 60}")
print(classification_report(y_test, y_pred_best, target_names=['Non-
Explicit (0)', 'Explicit (1)']))

# Confusion Matrix für das beste Modell
fig, ax = plt.subplots(figsize=(6, 5))
cm = confusion_matrix(y_test, y_pred_best)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=['Non-Explicit', 'Explicit'])
disp.plot(cmap=plt.cm.Blues, ax=ax)
ax.set_title(f'Confusion Matrix: {best_model_name}')
plt.tight_layout()
plt.show()
```

```
Bestes Modell: MLPClassifier + GridSearchCV (Punkt 2)
Test Accuracy: 0.7272
Trainingszeit: 1.4569 s
Inferenzzeit:  0.0030 s
```

```
==============================================================
Classification Report für MLPClassifier + GridSearchCV (Punkt 2):
==============================================================
                   precision    recall  f1-score   support

Non-Explicit (0)       0.74      0.88      0.81       590
    Explicit (1)       0.68      0.45      0.54       330

        accuracy                           0.73       920
       macro avg       0.71      0.67      0.67       920
    weighted avg       0.72      0.73      0.71       920
```



4.6 Diskussion

Accuracy: Die klassischen Modelle und die neuronalen Netze erreichen auf diesem Datensatz ziemlich ähnliche Werte. Das ist nicht überraschend, weil der Datensatz relativ klein ist und wir nur 9 numerische Features für eine binäre Klassifikation verwenden, da können auch einfache Modelle gut mithalten.

Trainingszeit: Hier sieht man den größten Unterschied. Logistic Regression oder Decision Tree brauchen Bruchteile von Sekunden, während die neuronalen Netze deutlich länger trainieren.

Keras braucht am längsten wegen der vielen Epochen. Rechnet man die Hyperparameter-Suche dazu, wird der Unterschied noch größer.

Inferenzzeit: Bei der Vorhersage sind alle Modelle schnell, weil das Testset klein ist. KNN ist etwas langsamer, weil es jedes Mal die Abstände zu allen Trainingsdaten berechnen muss.

Overfitting: Wenn Train Accuracy viel höher ist als Test Accuracy, hat das Modell die Daten auswendig gelernt. Decision Tree und Random Forest neigen dazu am meisten. Bei den neuronalen Netzen haben wir das mit Early Stopping und Dropout abgefangen.

https://studenthtlwrnac-my.sharepoint.com/:f:/g/personal/20210235_htlwrn_ac_at/IgBGq6cJ6JtmRI2i2Xz8pGesAe71VP4IwdY6Sp7hn6xLUAc?e=dC7l4Z